The AI Operating System, Part II: Tool Interface Layer — Morphisms, Security, and Capability Abstraction

Matthew Long
The YonedaAI Collaboration
YonedaAI Research Collective
Chicago, IL
matthew@yonedaai.com · https://yonedaai.com

March 10, 2026

Abstract

An AI Operating System mediates every interaction between intelligent agents and the external world through a *tool interface layer* that is analogous to the device-driver subsystem of a classical operating system. In this second paper of a five-part series, we develop a categorical framework in which tools are morphisms in a typed category Tool, capability tokens are representable functors $\mathrm{Hom}_{\mathbf{Tool}}(T, -)$, and sandboxing corresponds to passage to a quotient subcategory. We prove that tool composition inherits the associativity and identity axioms of the underlying category, that type-safe invocation preserves correctness through a functorial mapping from JSON Schema to algebraic types, and that sandbox isolation prevents information leakage under a formal adversarial model. The framework is instantiated concretely through an analysis of the Agent-Hero platform's three-tier tool registry—Builtin, Sandbox, and MCP (Model Context Protocol)—with its capability-based access control, contract-locked configurations, E2B ephemeral sandboxes, and per-invocation audit logging. We provide a complete implementation in Elixir/OTP that exploits BEAM process isolation as a natural realisation of categorical sandboxing, with GenServer-based registries, capability tokens as process-local state, and supervision trees that mirror the compositional hierarchy. Comparisons are drawn to Unix device models, Plan 9's everything-is-a-file philosophy, Capsicum capabilities, and seL4 verified microkernel security. The paper contains over thirty references spanning capability security, distributed systems, category theory, and the Model Context Protocol specification.

Keywords: AI operating system, tool interface, capability-based security, Model Context Protocol, category theory, morphism composition, Elixir/OTP, sandboxing, type safety

Contents

1 Introduction

The fundamental architectural insight of classical operating systems is that application code must never interact with hardware directly. Instead, a layer of *device drivers* interposes between user-space programs and peripheral devices, providing abstraction, multiplexing, isolation, and auditability (**?**). An application opens a file descriptor, reads and writes through a well-defined system-call interface, and the kernel routes those operations to the appropriate driver. The application need not know whether the underlying device is a local disk, a network socket, or a USB peripheral.

In an AI Operating System, the same architectural principle applies with even greater force. Agents—large language models with autonomy over their computational environment—must interact with external systems: databases, web APIs, file systems, code interpreters, and other agents. These interactions are mediated by *tools*: typed functions that accept structured input and return structured output, with well-defined security boundaries, execution constraints, and audit trails.

This paper develops a formal theory of the tool interface layer. Our contributions are:

1. A **categorical model** in which tools are morphisms in a category **Tool** whose objects are data types, capability tokens are representable functors, and sandboxing is a categorical quotient (Section **??**).
2. A **formal security architecture** based on capability-based access control, with proofs that sandbox isolation prevents information leakage and that contract-locked configurations form frozen subcategories (Section **??**).
3. A detailed analysis of the **Model Context Protocol** (MCP) as a universal tool interface satisfying a Yoneda-like universality property (Section **??**).
4. A **type-theoretic framework** for tool invocation in which JSON Schema to algebraic type conversion is a functor that preserves correctness (Section **??**).
5. A complete **Elixir/OTP implementation** that realises the categorical abstractions through BEAM process isolation (Section **??**).
6. **Case studies** drawn from the Agent-Hero production platform, including web scraping pipelines, sandboxed code execution, and multi-MCP server orchestration (Section **??**).

Relation to the series. This is Part II of a five-part series on the AI Operating System. Part I (**?**) established the kernel architecture—the process model and scheduling layer. The present paper builds on that foundation by defining the interface through which scheduled agent processes access external capabilities. Part III will address memory and persistence, Part IV the agent orchestration layer, and Part V the integration of all components into a unified operating system theory.

2 Background

2.1 Device Drivers and System Calls

The device driver model in Unix-like operating systems provides three critical properties (**?**):

1. **Abstraction.** Applications interact with a uniform file-descriptor interface regardless of the underlying device.
2. **Isolation.** A misbehaving driver cannot (in principle) corrupt unrelated kernel state, especially in microkernel designs.
3. **Multiplexing.** Multiple processes can share a single device through the driver's queuing and scheduling logic.

Plan 9 from Bell Labs extended this model to its logical conclusion: every resource—networks, graphics, even per-process name spaces—is presented as a file system (**?**). This "everything is a file" philosophy provides a universal interface that any program can consume without special-purpose client libraries. We argue that the AI OS tool layer should aspire to the same universality.

2.2 Capability-Based Security

Traditional access control uses access control lists (ACLs) attached to resources. Capability-based security inverts this: each subject holds unforgeable *capability tokens* that grant specific permissions on specific objects (**??**). Modern realisations include:

- **Capsicum** (FreeBSD): capability mode restricts a sandboxed process to only the file descriptors it was given at entry (**?**).
- **seL4**: a formally verified microkernel where every system call requires a capability argument (**?**).
- **WASI** (WebAssembly System Interface): capability-based access to host resources from sandboxed WebAssembly modules (**?**).
- **Macaroons**: decentralised capability tokens with contextual caveats for distributed systems (**?**).

The AI OS adapts capability-based security to the agent setting: each agent holds a set of capability tokens that determine which tools it may invoke, with what parameters, and at what rate.

2.3 Category Theory Preliminaries

We assume familiarity with basic category theory (**???**). We recall the essential definitions for self-containment.

Definition 2.1 (Category)**.** A *category* $\mathcal{C}$ consists of a collection $\mathrm{ob}(\mathcal{C})$ of objects, for each pair of objects A, B a set $\mathrm{Hom}_{\mathcal{C}}(A, B)$ of morphisms, an identity morphism $\mathrm{id}_A \in \mathrm{Hom}_{\mathcal{C}}(A, A)$ for each object A, and a composition operation $\circ : \mathrm{Hom}_{\mathcal{C}}(B, C) \times \mathrm{Hom}_{\mathcal{C}}(A, B) \to \mathrm{Hom}_{\mathcal{C}}(A, C)$ that is associative and unital.

Definition 2.2 (Functor)**.** A *functor* $F : \mathcal{C} \to \mathcal{D}$ maps objects to objects and morphisms to morphisms, preserving identity and composition: $F(\mathrm{id}_A) = \mathrm{id}_{F(A)}$ and $F(g \circ f) = F(g) \circ F(f)$.

Definition 2.3 (Representable Functor)**.** For an object T in a category $\mathcal{C}$, the *representable functor* $\mathrm{Hom}_{\mathcal{C}}(T, -) : \mathcal{C} \to \mathbf{Set}$ maps each object A to the set $\mathrm{Hom}_{\mathcal{C}}(T, A)$ and each morphism $f : A \to B$ to the post-composition function $f_* : \mathrm{Hom}_{\mathcal{C}}(T, A) \to \mathrm{Hom}_{\mathcal{C}}(T, B)$.

Definition 2.4 (Subobject Classifier)**.** In a topos $\mathcal{E}$, a *subobject classifier* is an object Ω together with a morphism $\top : 1 \to \Omega$ such that for every mono $m : S \hookrightarrow X$ there exists a unique characteristic morphism $\chi_m : X \to \Omega$ making the appropriate pullback square commute (**?**).

2.4 Erlang/OTP and the BEAM

The BEAM virtual machine (Erlang/Elixir runtime) provides lightweight processes with the following properties (**??**):

- **Process isolation**: each process has its own heap; no shared mutable state.
- **Message passing**: processes communicate only through asynchronous messages.
- **Supervision trees**: processes are organised hierarchically; a supervisor restarts failed children according to a declared strategy.
- **Hot code loading**: modules can be upgraded at runtime without stopping the system.

These properties make the BEAM an ideal substrate for implementing categorical sandboxing: each sandboxed tool execution becomes a BEAM process whose capability tokens are stored in process-local state, inaccessible to other processes.

3 Tool Model — Formal Definition

3.1 The Tool Category

Definition 3.1 (Tool Category **Tool**)**.** The *tool category* **Tool** is defined as follows:

- **Objects.** The objects of **Tool** are *data types*—structured schemas that describe the input and output of tool operations. Formally, $\mathrm{ob}(\mathbf{Tool}) = \{\tau \mid \tau \text{ is a JSON Schema type}\}$.
- **Morphisms.** A morphism $t : \tau_{\text{in}} \to \tau_{\text{out}}$ is a *tool operation* that accepts input of type τ_{in} and produces output of type τ_{out}.
- **Identity.** For each type τ, the identity morphism $\mathrm{id}_\tau : \tau \to \tau$ is the pass-through operation that returns its input unchanged.
- **Composition.** Given $t_1 : \tau_A \to \tau_B$ and $t_2 : \tau_B \to \tau_C$, the composite $t_2 \circ t_1 : \tau_A \to \tau_C$ is the tool pipeline that feeds the output of t_1 into t_2.

Theorem 3.2 (Tool Composition is Associative)**.** *For any composable morphisms $t_1 : \tau_A \to \tau_B$, $t_2 : \tau_B \to \tau_C$, and $t_3 : \tau_C \to \tau_D$ in* **Tool***, composition is associative:*

$$t_3 \circ (t_2 \circ t_1) = (t_3 \circ t_2) \circ t_1.$$

Proof. Tool operations are functions on typed data. Function composition is associative: for any input $x \in \tau_A$,

$$(t_3 \circ (t_2 \circ t_1))(x) = t_3((t_2 \circ t_1)(x)) = t_3(t_2(t_1(x))), \tag{1}$$
$$((t_3 \circ t_2) \circ t_1)(x) = (t_3 \circ t_2)(t_1(x)) = t_3(t_2(t_1(x))). \tag{2}$$

The two expressions are identical for all x, establishing associativity. Identity morphisms satisfy $\mathrm{id}_{\tau_B} \circ t_1 = t_1$ and $t_1 \circ \mathrm{id}_{\tau_A} = t_1$ by the same argument. Thus **Tool** satisfies the category axioms. □

$$\begin{tikzcd}
\tau_A \arrow[r, "t_1"] \arrow[rr, bend left, "t_2 \circ t_1"] \arrow[rrr, bend left=60, "t_3 \circ t_2 \circ t_1"] & \tau_B \arrow[r, "t_2"] & \tau_C \arrow[r, "t_3"] & \tau_D
\end{tikzcd}$$

Figure 1: Morphism composition in **Tool**. Associativity ensures that the order of bracketing in a pipeline of tool operations is immaterial.

3.2 Tool Tiers as Subcategories

The Agent-Hero platform organises tools into three tiers, each forming a subcategory of **Tool**:

Definition 3.3 (Three-Tier Decomposition)**.** The tool category decomposes as:

$$\mathbf{Tool} = \mathbf{Tool}_{\text{Builtin}} \cup \mathbf{Tool}_{\text{Sandbox}} \cup \mathbf{Tool}_{\text{MCP}}$$

where:

1. $\mathbf{Tool}_{\text{Builtin}}$ contains statically defined, trusted operations: `web-search`, `text-transform`, `web-scrape`, `pdf-parse`, `generate-document`, `spreadsheet-read`.

2. $\mathbf{Tool}_{\text{Sandbox}}$ contains operations that execute arbitrary code in an isolated environment: `git-clone`, `npm-run`, `code-exec`, `file-ops`, `python-exec`, `shell-exec`.

3. $\mathbf{Tool}_{\text{MCP}}$ contains runtime-discovered tools provided by external MCP servers, namespaced as `${serverName}__${toolName}`.

Each tier has a different trust level, which we formalise through capability restrictions in Section **??**.

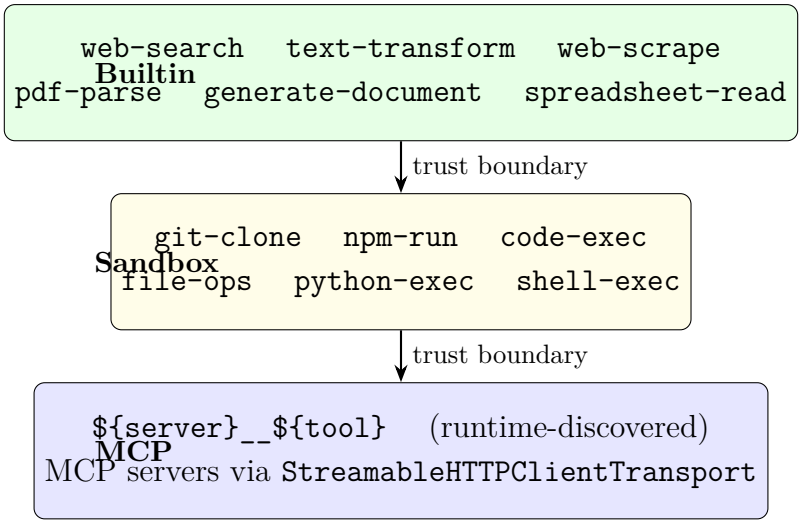

Figure 2: Three-tier tool registry in Agent-Hero. Trust decreases from top to bottom; capability requirements increase correspondingly.

3.3 Capability Tokens as Representable Functors

Definition 3.4 (Capability Token)**.** Let T be an object of **Tool** representing a specific tool's input type. A *capability token* for T is the representable functor

$$\Cap_T := \mathrm{Hom}_{\mathbf{Tool}}(T, -) : \mathbf{Tool} \to \mathbf{Set}.$$

For each output type τ, $\Cap_T(\tau) = \mathrm{Hom}_{\mathbf{Tool}}(T, \tau)$ is the set of tool operations accessible from T that produce values of type τ.

The Yoneda lemma (**?**) guarantees that each capability token faithfully represents the tool:

Proposition 3.5 (Capability Faithfulness via Yoneda)**.** *For any functor $F : \mathbf{Tool} \to \mathbf{Set}$, there is a natural bijection*

$$\mathit{Nat}(\mathrm{Hom}_{\mathbf{Tool}}(T, -), F) \cong F(T).$$

In particular, taking $F = \mathrm{Hom}_{\mathbf{Tool}}(T', -)$, we obtain $\mathrm{Hom}_{\mathbf{Tool}}(T, T') \cong \mathit{Nat}(\mathrm{Hom}_{\mathbf{Tool}}(T', -), \mathrm{Hom}_{\mathbf{Tool}}(T,$ Two tools are indistinguishable by their capability tokens if and only if they are isomorphic in **Tool**.

Remark 3.6. In the implementation, a capability token is concretised as a signed, scoped data structure:

$$\mathrm{token} = (\mathrm{agent_id}, \mathrm{tool_id}, \mathrm{permissions}, \mathrm{rate_limit}, \mathrm{expiry}, \mathrm{signature}).$$

The functor interpretation ensures that the bearer of a token can only access morphisms (tool operations) that the token's scope includes.

4 Security Architecture

4.1 Capability-Based Access Control

Definition 4.1 (Capability System)**.** A *capability system* for **Tool** is a triple $(\mathcal{A}, \Gamma, \vdash)$ where:

- $\mathcal{A}$ is a set of *agents*,
- $\Gamma : \mathcal{A} \to \mathcal{P}(\mathrm{Mor}(\mathbf{Tool}))$ assigns to each agent a set of permitted morphisms (its capability set),
- $\vdash$ is a judgement: $a \vdash t$ means agent a is authorised to invoke tool t.

The judgement is defined by: $a \vdash t \iff t \in \Gamma(a)$.

Axiom 4.2 (Principle of Least Authority)**.** For each agent $a \in \mathcal{A}$ and task ϕ, the capability set $\Gamma(a)$ contains only those morphisms necessary to complete ϕ:

$$\Gamma(a) = \{t \in \mathrm{Mor}(\mathbf{Tool}) \mid \phi \text{ requires } t\}.$$

In the Agent-Hero system, the Principle of Least Authority is enforced through *contract-locked tool configurations*: when an agent execution begins, the set of available tools is frozen and cannot be modified during the run. This prevents privilege escalation through dynamic tool discovery.

4.2 Sandboxing as Categorical Quotient

Definition 4.3 (Sandbox Subcategory)**.** Given a capability set $\Gamma(a)$ for agent a, the *sandbox* of a is the subcategory

$$\mathbf{Sand}_a := \mathbf{Tool}|_{\Gamma(a)}$$

where $\mathrm{ob}(\mathbf{Sand}_a) = \{\tau \in \mathrm{ob}(\mathbf{Tool}) \mid \exists t \in \Gamma(a).\ \mathrm{src}(t) = \tau \text{ or } \mathrm{tgt}(t) = \tau\}$ and $\mathrm{Mor}(\mathbf{Sand}_a) = \Gamma(a) \cup \{\mathrm{id}_\tau \mid \tau \in \mathrm{ob}(\mathbf{Sand}_a)\}$.

Definition 4.4 (Information Leakage)**.** Let $\mathbf{Sand}_a$ and $\mathbf{Sand}_b$ be sandboxes for agents a and b. An *information leak* from a to b is a morphism $\ell : \tau_a \to \tau_b$ where $\tau_a \in \mathrm{ob}(\mathbf{Sand}_a) \setminus \mathrm{ob}(\mathbf{Sand}_b)$ and $\tau_b \in \mathrm{ob}(\mathbf{Sand}_b)$, and $\ell \notin \Gamma(a) \cup \Gamma(b)$.

Theorem 4.5 (Sandbox Isolation)**.** *If* $\mathbf{Sand}_a$ *and* $\mathbf{Sand}_b$ *are sandboxes with disjoint capability sets,* $\Gamma(a) \cap \Gamma(b) = \emptyset$*, and the only morphisms available to each agent are those in their respective sandboxes, then no information leak exists from* a *to* b*.*

Proof. Suppose for contradiction that a leak $\ell : \tau_a \to \tau_b$ exists. By Definition **??**, $\ell \notin \Gamma(a)$ and $\ell \notin \Gamma(b)$. Since agent a can only invoke morphisms in $\Gamma(a)$ and agent b can only invoke morphisms in $\Gamma(b)$, neither agent can invoke ℓ. Therefore ℓ cannot be executed, contradicting the assumption that it constitutes a leak. More generally, any composite path $\tau_a \to \cdots \to \tau_b$ must traverse at least one morphism outside both $\Gamma(a)$ and $\Gamma(b)$, which neither agent can invoke.

In the BEAM implementation, this corresponds to the fact that BEAM processes have isolated heaps: process P_a implementing $\mathbf{Sand}_a$ cannot read the memory of process P_b implementing $\mathbf{Sand}_b$, and all communication must go through explicit message passing mediated by the capability system. □

4.3 Contract-Locked Configurations as Frozen Subcategories

Definition 4.6 (Frozen Subcategory)**.** A subcategory $\mathcal{F} \hookrightarrow \mathbf{Tool}$ is *frozen* if no new morphisms or objects can be added to $\mathcal{F}$ after its creation. Formally, $\mathcal{F}$ is equipped with a timestamp t_0 such that for all $t > t_0$, $\mathrm{ob}(\mathcal{F}_t) = \mathrm{ob}(\mathcal{F}_{t_0})$ and $\mathrm{Mor}(\mathcal{F}_t) = \mathrm{Mor}(\mathcal{F}_{t_0})$.

This corresponds directly to the Agent-Hero implementation, where tool configurations are locked at execution start:

```
// Tool set is frozen at execution start
const frozenTools = Object.freeze(
  resolveToolsForAgent(agent, contract)
);

// Subsequent attempts to modify tools throw
// Cannot add properties to a frozen object
async function executeAgent(agent, task) {
  const tools = frozenTools; // immutable reference
  // tools cannot be modified during execution
  return await runAgentLoop(agent, task, tools);
}
```

Listing 1: Contract-locked tool configuration (simplified from Agent-Hero).

4.4 URL Validation as Morphism Filtering

A critical security concern is preventing agents from accessing internal resources. The Agent-Hero platform implements URL validation that blocks access to localhost and internal IP ranges:

Definition 4.7 (Safe Morphism Filter)**.** Let $\mathbf{Tool}_{\text{web}}$ be the subcategory of web-accessing tools. The *safe morphism filter* σ is an endofunctor $\sigma : \mathbf{Tool}_{\text{web}} \to \mathbf{Tool}_{\text{web}}$ defined by:

$$\sigma(t)(x) = \begin{cases} t(x) & \text{if } \mathrm{url}(x) \notin \mathrm{BLOCKED} \\ \perp & \text{otherwise} \end{cases}$$

where $\mathrm{BLOCKED} = \{\texttt{localhost}, \texttt{127.0.0.0/8}, \texttt{169.254.0.0/16}, \texttt{10.0.0.0/8}, \texttt{172.16.0.0/12}, \texttt{192.16}$

$$\begin{tikzcd}
\tau_{\text{URL}} \arrow[r, "t_{\text{fetch}}"] \arrow[d, "\sigma"'] & \tau_{\text{HTML}} \\
\tau_{\text{URL}} \arrow[r, "t_{\text{safe-fetch}}"'] & \tau_{\text{HTML}} \cup \{\perp\} \arrow[u, hook]
\end{tikzcd}$$

Figure 3: URL validation as a morphism filter. The safe-fetch tool either produces valid HTML or rejects the request ($\perp$) for blocked addresses.

5 Protocol Design: The Model Context Protocol

5.1 MCP as a Universal Tool Interface

The Model Context Protocol (MCP) (**?**) defines a standard interface for language models to discover and invoke tools provided by external servers. In the AGENT-HERO platform, MCP integration uses the `@modelcontextprotocol/sdk` library with `StreamableHTTPClientTransport` for efficient HTTP-based communication and Bearer token authentication.

Definition 5.1 (MCP Interface)**.** An *MCP server* S exposes:

1. A *tool listing* endpoint returning $\{(n_i, \sigma_i)\}_{i \in I}$ where n_i is a tool name and σ_i is its JSON Schema.
2. An *invocation* endpoint that, given a tool name n_i and input conforming to σ_i, returns a structured result.
3. A *resource listing* endpoint for data resources the server provides.

Definition 5.2 (MCP Namespacing)**.** Given an MCP server with name s providing tools $\{t_1, \ldots, t_k\}$, the namespaced tool identifiers are $\{s__t_1, \ldots, s__t_k\}$. This prevents naming collisions when multiple MCP servers are connected simultaneously.

Construction 5.3 (Parallel MCP Discovery)**.** The AGENT-HERO platform connects to multiple MCP servers in parallel using `Promise.allSettled()`, ensuring that a slow or failing server does not block discovery from other servers:

```
const connections = await Promise.allSettled(
  mcpServers.map(async (server) => {
    const transport = new StreamableHTTPClientTransport(
      new URL(server.url),
      { headers: { Authorization: `Bearer ${server.token}` } }
    );
    const client = new Client({ name: "agent-hero", version: "1.0" });
    await client.connect(transport);
    const { tools } = await client.listTools();
    return tools.map(t => ({
      name: `${server.name}__${t.name}`,
      description: t.description,
      schema: t.inputSchema,
    }));
  })
);
```

Listing 2: Parallel MCP server connection (from Agent-Hero).

5.2 Universality of MCP

We now establish that MCP satisfies a universality property analogous to the Yoneda lemma.

Definition 5.4 (Tool Interface Functor)**.** A *tool interface* is a functor $I : \mathbf{Tool} \to \mathbf{Set}$ that maps each tool to its observable behaviour: for a morphism $t : \tau_A \to \tau_B$, $I(t)$ is the function from inputs of type τ_A to outputs of type τ_B, as observed through the interface.

Proposition 5.5 (MCP Universality)**.** *The MCP interface I_{MCP} is universal among tool interfaces in the following sense: for any tool interface $I : \mathbf{Tool} \to \mathbf{Set}$, there exists a unique natural transformation $\eta : I_{MCP} \Rightarrow I$ such that the following diagram commutes for every tool t:*

$$\begin{tikzcd} \mathbf{Tool} \arrow[r, "I_{MCP}"] \arrow[dr, "I"'] & \mathbf{Set} \arrow[d, "\eta"] \\ & \mathbf{Set} \end{tikzcd}$$

This follows from the fact that MCP's JSON Schema-based tool description is sufficient to reconstruct any type-safe invocation pattern: the schema determines the input type, the invocation protocol determines the function, and the response format determines the output type. Any tool interface that respects typing must factor through MCP's schema-driven description.

Proof sketch. The key observation is that MCP's tool description is *schema-complete*: every tool operation can be described by a triple (name, input JSON Schema, output JSON Schema). Given any alternative interface I, we construct η componentwise: for each type τ, $\eta_\tau : I_{\text{MCP}}(\tau) \to I(\tau)$ maps an MCP-described operation to its I-representation. Uniqueness follows from the determinism of JSON Schema interpretation—two descriptions that agree on schema must agree on their type-level behaviour. Naturality is verified by checking that schema-preserving tool composition in MCP commutes with the corresponding composition in I. □

6 Type-Safe Tool Invocation

6.1 JSON Schema to Algebraic Types

The Agent-Hero platform converts JSON Schema definitions to Zod validators for runtime type safety. This conversion is a functor:

Definition 6.1 (Schema Functor)**.** The *schema functor* $\Sigma : \mathbf{JSONSchema} \to \mathbf{Zod}$ maps:

- JSON Schema type objects to Zod schema objects.
- Schema refinements (subtyping relations) to Zod refinement compositions.

where **JSONSchema** is the category of JSON Schema types with subtyping as morphisms and **Zod** is the category of Zod validators with validator composition as morphisms.

The conversion process is exemplified in the Vercel AI SDK v6 integration:

```
import { tool } from 'ai';
import { zodSchema } from 'ai/schema';
import { z } from 'zod';

// Static tool definition with Zod schema
const webSearchTool = tool({
  description: 'Search the web for information',
  parameters: zodSchema(z.object({
```

```
    query: z.string().describe('Search query'),
    maxResults: z.number().int().min(1).max(20).default(10),
  })),
  execute: async ({ query, maxResults }) => {
    // Type-safe: query is string, maxResults is number
    return await performWebSearch(query, maxResults);
  },
});

// MCP-discovered tool: JSON Schema -> Zod conversion
function mcpToolToVercelTool(mcpTool: MCPTool) {
  return tool({
    description: mcpTool.description,
    parameters: zodSchema(
      jsonSchemaToZod(mcpTool.inputSchema) // functor application
    ),
    execute: async (input) => {
      return await mcpClient.callTool(mcpTool.name, input);
    },
  });
}
```

Listing 3: JSON Schema to Zod conversion for tool inputs (Agent-Hero pattern).

6.2 Correctness Preservation

Theorem 6.2 (Type-Safe Composition Preserves Correctness)**.** *Let $t_1 : \tau_A \to \tau_B$ and $t_2 : \tau_B \to \tau_C$ be tool morphisms with associated Zod validators $v_A = \Sigma(\tau_A)$, $v_B = \Sigma(\tau_B)$, and $v_C = \Sigma(\tau_C)$. If t_1 and t_2 individually preserve type correctness (i.e., valid inputs produce valid outputs), then the composite $t_2 \circ t_1$ preserves type correctness.*

Proof. Assume $x \in v_A$ (i.e., x passes validation by v_A). Since t_1 preserves correctness, $t_1(x) \in v_B$. Since t_2 preserves correctness and $t_1(x) \in v_B$, $t_2(t_1(x)) \in v_C$. Therefore $(t_2 \circ t_1)(x) \in v_C$ for all $x \in v_A$.

More formally, the schema functor Σ preserves composition: $\Sigma(t_2 \circ t_1) = \Sigma(t_2) \circ \Sigma(t_1)$. The correctness predicate $\text{Correct}(t) \equiv \forall x.\ x \in \Sigma(\text{src}(t)) \implies t(x) \in \Sigma(\text{tgt}(t))$ is preserved under composition by transitivity. □

$$\begin{tikzcd}
\textbf{JSONSchema} \arrow[r, "\Sigma"] \arrow[d, "t_1 \circ t_2"'] & \textbf{Zod} \arrow[d, "\Sigma(t_1) \circ \Sigma(t_2)"] \\
\textbf{JSONSchema} \arrow[r, "\Sigma"'] & \textbf{Zod}
\end{tikzcd}$$

Figure 4: The schema functor Σ commutes with tool composition, ensuring that type safety is preserved through pipelines.

6.3 Runtime Validation Pipeline

The complete validation pipeline for a tool invocation proceeds as follows:

Algorithm 1: Type-safe tool invocation with capability checking and audit.

Input : Agent a, tool morphism $t : \tau_A \to \tau_B$, input value x
Output: Result $y \in \tau_B$ or error ϵ

```
if t ∉ Γ(a) then
    return ε_unauthorized;
end
v_A ← Σ(τ_A);
if x ∉ v_A then
    return ε_validation;
end
y ← execute(t, x) ;                    // in sandbox if t ∈ Tool_Sandbox
v_B ← Σ(τ_B);
if y ∉ v_B then
    return ε_output_validation;
end
audit_log(a, t, x, y, timestamp);
return y;
```

7 Elixir/OTP Implementation

The categorical abstractions of Sections **??**–**??** map naturally onto Elixir/OTP primitives. We describe the key components of a production implementation.

7.1 Architecture Overview

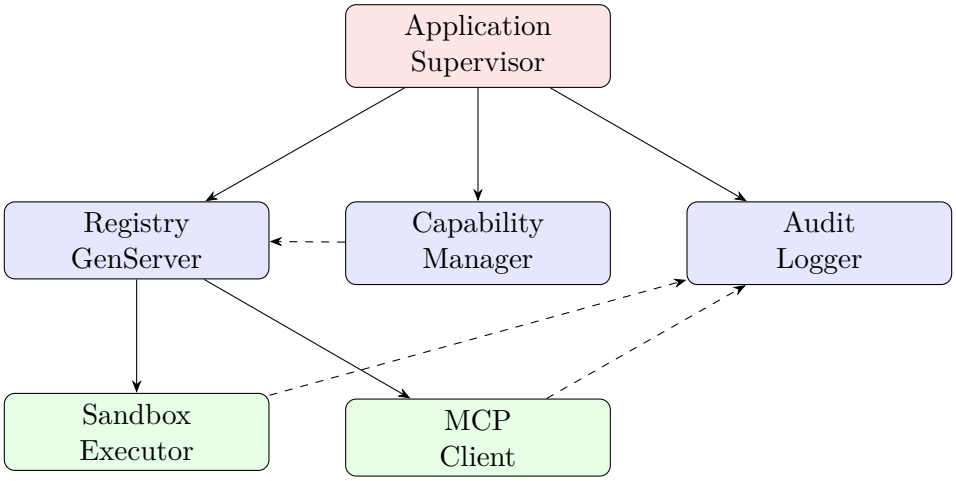

Figure 5: OTP supervision tree for the tool interface layer. Solid arrows represent supervision; dashed arrows represent inter-process communication.

7.2 Tool Registry GenServer

The tool registry maintains the three-tier tool database as a GenServer with the following state:

```
defmodule ToolInterface.Registry do
  use GenServer
  require Logger

  @type tool_tier :: :builtin | :sandbox | :mcp
  @type tool_spec :: %{
    name: String.t(),
    tier: tool_tier(),
    description: String.t(),
    input_schema: map(),
    output_schema: map(),
    execute: (map() -> {:ok, any()} | {:error, term()})
  }

  defstruct builtin: %{}, sandbox: %{}, mcp: %{}, frozen: false

  def start_link(opts) do
    GenServer.start_link(__MODULE__, opts, name: __MODULE__)
  end

  def init(_opts) do
    state = %__MODULE__{
      builtin: load_builtin_tools(),
      sandbox: load_sandbox_tools(),
      mcp: %{}
    }
    {:ok, state}
  end

  def handle_call({:register_mcp, server, tools}, _from, state) do
    if state.frozen do
      {:reply, {:error, :frozen}, state}
    else
      namespaced = for t <- tools, into: %{} do
        {"#{server}__#{t.name}", t}
      end
      {:reply, :ok, %{state | mcp: Map.merge(state.mcp, namespaced)}}
    end
  end

  def handle_call(:freeze, _from, state) do
    {:reply, :ok, %{state | frozen: true}}
  end
end
```

Listing 4: Tool registry GenServer (excerpt).

The `:freeze` operation implements contract-locked configurations (Definition **??**): once frozen, no new tools can be registered.

7.3 Capability Token System

Capability tokens are implemented as signed, scoped structures stored in process-local state:

```
defmodule ToolInterface.Capability do
  @type t :: %__MODULE__{
    agent_id: String.t(),
    tool_id: String.t(),
    permissions: [:invoke | :inspect | :compose],
    rate_limit: pos_integer(),
    expires_at: DateTime.t(),
    signature: binary()
  }

  defstruct [:agent_id, :tool_id, :permissions,
             :rate_limit, :expires_at, :signature]

  @spec authorize(t(), String.t()) ::
    {:ok, t()} | {:error, :unauthorized | :expired | :rate_limited}
  def authorize(token, tool_id) do
    cond do
      token.tool_id != tool_id -> {:error, :unauthorized}
      DateTime.compare(DateTime.utc_now(), token.expires_at) == :gt ->
        {:error, :expired}
      not valid_signature?(token) -> {:error, :unauthorized}
      true -> {:ok, token}
    end
  end
end
```

Listing 5: Capability token module (excerpt).

7.4 Sandbox Execution

Sandbox tools execute in isolated BEAM processes with restricted capabilities, mirroring the E2B sandbox model with ephemeral, time-bounded execution:

```
defmodule ToolInterface.Sandbox do
  @timeout 600_000  # 10 minutes, matching E2B

  @spec execute(map(), map(), keyword()) ::
    {:ok, any()} | {:error, term()}
  def execute(tool_spec, input, opts \\ []) do
    timeout = Keyword.get(opts, :timeout, @timeout)
    parent = self()

    {pid, ref} = spawn_monitor(fn ->
      # Process runs in isolation - separate heap
      result = try do
        tool_spec.execute.(input)
      rescue
        e -> {:error, Exception.message(e)}
      end
      send(parent, {:sandbox_result, self(), result})
    end)
```

```

    receive do
      {:sandbox_result, ^pid, result} ->
        Process.demonitor(ref, [:flush])
        result
      {:DOWN, ^ref, :process, ^pid, reason} ->
        {:error, {:sandbox_crashed, reason}}
    after
      timeout ->
        Process.exit(pid, :kill)
        receive do
          {:DOWN, ^ref, :process, ^pid, _} -> :ok
        end
        {:error, :timeout}
    end
  end
end
```

Listing 6: Sandbox executor (excerpt).

This pattern directly implements sandbox isolation (Theorem **??**): the spawned process has its own heap, cannot access the parent's state, and is killed if it exceeds the timeout.

7.5 MCP Client

The MCP client handles tool discovery and invocation through HTTP transport:

```
defmodule ToolInterface.MCPClient do
  use GenServer

  defstruct [:server_name, :url, :token, :tools]

  def start_link(opts) do
    GenServer.start_link(__MODULE__, opts)
  end

  def init(opts) do
    state = %__MODULE__{
      server_name: opts[:name],
      url: opts[:url],
      token: opts[:token],
      tools: %{}
    }
    {:ok, state, {:continue, :discover}}
  end

  def handle_continue(:discover, state) do
    case discover_tools(state.url, state.token) do
      {:ok, tools} ->
        namespaced = for t <- tools, into: %{} do
          {"#{state.server_name}__#{t["name"]}", t}
        end
        ToolInterface.Registry.register_mcp(
          state.server_name, tools
        )
        {:noreply, %{state | tools: namespaced}}
```

```
      {:error, reason} ->
        Logger.warning("MCP discovery failed: #{reason}")
        {:noreply, state}
    end
  end
end
```

Listing 7: MCP client (excerpt).

7.6 Audit Logger

Per-invocation audit logging captures timestamp, agent identity, tool identity, input, output, and execution duration:

```
defmodule ToolInterface.Audit do
  use GenServer

  @type entry :: %{
    timestamp: DateTime.t(),
    agent_id: String.t(),
    tool_id: String.t(),
    input: map(),
    output: any(),
    duration_ms: non_neg_integer(),
    status: :ok | :error
  }

  def log_invocation(agent_id, tool_id, input, output,
                     duration_ms, status) do
    GenServer.cast(__MODULE__, {
      :log,
      %{
        timestamp: DateTime.utc_now(),
        agent_id: agent_id,
        tool_id: tool_id,
        input: input,
        output: output,
        duration_ms: duration_ms,
        status: status
      }
    })
  end

  def handle_cast({:log, entry}, state) do
    Logger.info("AUDIT: #{entry.agent_id} -> #{entry.tool_id} " <>
                "[#{entry.status}] #{entry.duration_ms}ms")
    {:noreply, %{state | entries: [entry | state.entries]}}
  end
end
```

Listing 8: Audit logger (excerpt).

8 Case Studies

8.1 Web Scraping Tool Chain

Consider an agent tasked with extracting structured data from a web page. The tool chain composes three morphisms in $\mathbf{Tool}_{\text{Builtin}}$:

1. $t_{\text{search}} : \tau_{\text{Query}} \to \tau_{\text{URLs}}$ — `web-search` finds relevant URLs.
2. $t_{\text{scrape}} : \tau_{\text{URL}} \to \tau_{\text{HTML}}$ — `web-scrape` retrieves page content.
3. $t_{\text{transform}} : \tau_{\text{HTML}} \to \tau_{\text{Structured}}$ — `text-transform` extracts structured data.

The composite $t_{\text{transform}} \circ t_{\text{scrape}} \circ t_{\text{search}}$ is a morphism $\tau_{\text{Query}} \to \tau_{\text{Structured}}$. By Theorem **??**, the order of composition bracketing is immaterial. By Theorem **??**, type safety propagates through the chain.

The URL validation filter (Definition **??**) ensures that t_{scrape} cannot access internal resources. In the Agent-Hero system, this is enforced at the tool level: before any HTTP request, the URL is checked against the blocked list (no `localhost` in production, no `169.254.x.x` link-local addresses).

$$\begin{tikzcd}
\tau_{\text{Query}} \arrow[r, "t_{\text{search}}"] & \tau_{\text{URLs}} \arrow[r, "t_{\text{scrape}}"] \arrow[d, dashed, "\sigma"] & \tau_{\text{HTML}} \arrow[r, "t_{\text{transform}}"] & \tau_{\text{Structured}} \\
& \tau_{\text{URLs}}^{\text{safe}} \arrow[ur, "t_{\text{scrape}}^{\text{safe}}"'] & &
\end{tikzcd}$$

Figure 6: Web scraping pipeline with URL validation filter σ. Only safe URLs proceed to the scrape step.

8.2 Code Execution Sandbox

The code execution sandbox tools (`code-exec`, `python-exec`, `shell-exec`) operate in $\mathbf{Tool}_{\text{Sandbox}}$ with the strictest isolation requirements. In the Agent-Hero system, these tools are executed in E2B sandboxes—ephemeral virtual environments with a 10-minute timeout:

1. The agent requests code execution with a capability token authorising `code-exec`.
2. The capability system validates the token (Algorithm **??**, lines 1–2).
3. A fresh E2B sandbox is provisioned—an ephemeral environment with no access to the host file system or network (beyond whitelisted endpoints).
4. The code is executed inside the sandbox. If the execution exceeds 10 minutes, the sandbox is terminated.
5. The result (stdout, stderr, exit code) is captured, validated against the output schema, and returned to the agent.
6. An audit entry is logged with the full execution trace.

Categorically, the E2B sandbox instantiates $\mathbf{Sand}_a$ (Definition **??**). The ephemeral nature ensures that no state leaks between executions: each sandbox is a fresh instance of the restricted subcategory.

```
import { Sandbox } from '@e2b/code-interpreter';

async function executeSandboxed(code: string, language: string) {
  const sandbox = await Sandbox.create({
    timeoutMs: 10 * 60 * 1000, // 10 minutes
  });
  try {
    const result = await sandbox.runCode(code, { language });
    return {
      stdout: result.logs.stdout,
      stderr: result.logs.stderr,
      exitCode: result.exitCode,
    };
  } finally {
    await sandbox.close(); // ephemeral: always clean up
  }
}
```

Listing 9: E2B sandbox execution pattern (from Agent-Hero).

8.3 Multi-MCP Server Orchestration

A production deployment may connect to multiple MCP servers simultaneously. The Agent-Hero platform handles this through parallel connection with fault isolation:

1. The platform maintains a list of configured MCP servers, each with a URL and Bearer token.
2. At agent execution start, `Promise.allSettled()` connects to all servers in parallel (Construction **??**).
3. Each successful connection contributes its tools to $\mathbf{Tool}_{\mathrm{MCP}}$, namespaced to prevent collisions.
4. Failed connections are logged but do not block execution—the agent proceeds with whatever tools were successfully discovered.
5. The combined tool set is frozen (Definition **??**), establishing the contract for the execution run.

In the Elixir implementation, each MCP server connection is a separate GenServer process. Parallel discovery is achieved via `Task.async_stream/3`:

```
def discover_all(servers) do
  servers
  |> Task.async_stream(
    fn server ->
      case MCPClient.discover(server.url, server.token) do
        {:ok, tools} -> {:ok, server.name, tools}
        error -> error
```

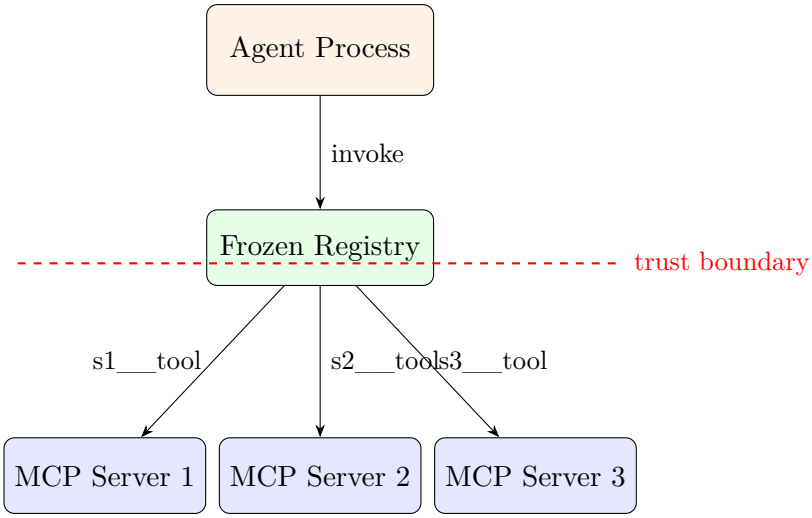

Figure 7: Multi-MCP server orchestration. The frozen registry mediates between the agent and multiple external MCP servers, with namespaced tool identifiers.

```
      end
    end,
    max_concurrency: 10,
    timeout: 30_000
  )
  |> Enum.reduce([], fn
    {:ok, {:ok, name, tools}}, acc -> [{name, tools} | acc]
    {:ok, {:error, _}}, acc -> acc
    {:exit, _}, acc -> acc
  end)
end
```

Listing 10: Parallel MCP discovery in Elixir.

9 Audit and Observability

9.1 Per-Invocation Trace Logging

Every tool invocation in the AI OS produces an audit record. Formally:

Definition 9.1 (Audit Functor)**.** The *audit functor* $\mathbf{Aud} : \mathbf{Tool} \to \mathbf{Log}$ maps each tool invocation to a log entry:

$$\mathbf{Aud}(t, x) = (\text{timestamp}, \text{agent_id}, t, x, t(x), \Delta t, \text{status})$$

where Δt is the execution duration and status $\in \{\text{ok}, \text{error}\}$. The functor preserves composition: $\mathbf{Aud}(t_2 \circ t_1, x) = \mathbf{Aud}(t_2, t_1(x)) \cdot \mathbf{Aud}(t_1, x)$ where $\cdot$ is log concatenation.

Proposition 9.2 (Audit Completeness)**.** *If every tool morphism $t \in \text{Mor}(\mathbf{Tool})$ passes through Algorithm* **??**, *then* **Aud** *captures every externally-observable action of every agent.*

Proof. By construction, Algorithm **??** is the sole invocation path for tool morphisms. The algorithm's final step (line 10) invokes audit_log, which applies **Aud**. Since no tool invocation bypasses this algorithm (enforced by the capability system's mediation of all tool access), every invocation produces an audit record. □

9.2 Cost Attribution

Each audit record can be extended with cost information, enabling per-tool, per-agent cost tracking:

Definition 9.3 (Cost Extension)**.** The *cost-extended audit functor* $\mathbf{Aud}^{\$} : \mathbf{Tool} \to \mathbf{Log}^{\$}$ augments each record with:

$$\text{cost}(t, x) = \text{base_cost}(t) + \text{compute_cost}(\Delta t) + \text{api_cost}(t, x)$$

where base_cost is the per-invocation charge, compute_cost is proportional to execution time, and api_cost accounts for third-party API charges (e.g., web search API costs).

This directly supports the credit-based billing model in Agent-Hero, where agents consume credits proportional to their tool usage, and operators are compensated based on the tools their agents provide.

9.3 Observability Stack

The audit system integrates with standard observability tools:

- **Structured logging**: Each audit entry is a JSON object, indexed by agent ID, tool ID, and timestamp.
- **Metrics**: Prometheus-compatible metrics on invocation count, latency percentiles, and error rates per tool tier.
- **Tracing**: OpenTelemetry spans for each tool invocation, with parent-child relationships reflecting tool composition.
- **Alerting**: Threshold-based alerts on error rates, latency spikes, and capability violation attempts.

In the Elixir implementation, the audit GenServer handles logging asynchronously via `GenServer.cast/2`, ensuring that audit overhead does not impact tool invocation latency. Periodic flushing to persistent storage (database or object storage) ensures durability.

10 Discussion

10.1 Comparison to Unix Device Model

The Unix device model (**?**) provides a useful comparison framework. In Unix, device drivers are accessed through file descriptors via `open()`, `read()`, `write()`, and `ioctl()` system calls. The AI OS tool layer is analogous:

Unix	**AI OS**
File descriptor	Capability token
Device driver	Tool implementation
System call	Tool invocation
`/dev` directory	Tool registry
Major/minor numbers	Tool tier + tool ID
`ioctl` parameters	Tool input schema
Kernel isolation	Sandbox subcategory
`open` permission check	Capability authorisation

The key difference is that the AI OS must handle *dynamic* tool discovery (through MCP) whereas Unix device discovery is largely static (though hot-plugging adds some dynamism). The contract-locking mechanism bridges this gap: tools are discovered dynamically but frozen at execution time, providing the stability guarantees of static configuration.

10.2 Comparison to Plan 9

Plan 9's "everything is a file" philosophy (**?**) suggests that the AI OS should present all tools through a uniform interface. MCP achieves this to a significant degree: every tool, regardless of its implementation, is invoked through the same protocol with the same schema-based input/output contract. The MCP universality proposition (Proposition **??**) formalises this uniformity.

However, Plan 9 goes further by allowing *per-process name spaces*: each process sees a different file system tree. The AI OS analogue is per-agent sandboxing: each agent's sandbox ($\mathbf{Sand}_a$) presents a different view of the tool category, restricted to the agent's capabilities. This is a richer model than Plan 9's flat file namespace, as it supports typed, composable operations rather than byte streams.

10.3 Comparison to seL4 and Capsicum

The seL4 microkernel (**?**) provides formally verified capability-based security. Our capability token model (Definition **??**) is inspired by seL4's capability system, but differs in several ways:

1. **Scope**: seL4 capabilities grant access to kernel objects (endpoints, pages, CNodes). Our capabilities grant access to tool morphisms.

2. **Delegation**: seL4 supports capability delegation and derivation. Our model supports time-bounded tokens but does not currently support agent-to-agent delegation (a direction for future work).

3. **Verification**: seL4's properties are machine-verified in Isabelle/HOL. Our theorems are proved mathematically; formal machine verification is future work.

Capsicum (**?**) provides a more pragmatic model: once a process enters capability mode, it can only access resources through file descriptors it already holds. Our contract-locked configuration (Definition **??**) is directly analogous: once execution begins, the tool set is frozen, and no new capabilities can be acquired.

10.4 Security Limitations

Our formal model has several limitations that must be acknowledged:

1. **Side channels**: The information leakage theorem (Theorem **??**) prevents *direct* data flow between sandboxes but does not address timing side channels or resource-usage side channels.

2. **Tool implementation trust**: We assume tool implementations are correct with respect to their schemas. A malicious tool implementation that ignores its type contract could violate type-safety guarantees.

3. **MCP server trust**: MCP-discovered tools rely on the security of external MCP servers. A compromised server could provide tools with misleading descriptions or malicious implementations.

4. **Prompt injection**: Tools that process external content (e.g., `web-scrape`) may return data containing prompt injection attacks. The type system validates structure but not semantic content.

10.5 Scalability Considerations

The BEAM virtual machine provides excellent scalability properties for the tool interface layer:

- **Concurrent tool invocations**: The BEAM scheduler can handle millions of lightweight processes, allowing massive concurrent tool execution.
- **Fault isolation**: A crashing tool execution affects only its own process; the registry and other tool executions are unaffected.
- **Back-pressure**: GenServer-based registries naturally provide back-pressure through message queue depth monitoring.
- **Distribution**: OTP distribution allows the tool registry to span multiple nodes, with capability tokens validated locally.

10.6 Future Directions

Several extensions of this work are planned:

1. **Capability delegation**: Allowing agents to delegate subsets of their capabilities to sub-agents, with formal guarantees that delegated capabilities cannot exceed the delegator's authority.

2. **Dependent type integration**: Moving beyond JSON Schema to dependent types (**?**), enabling schemas where the output type depends on the input value.

3. **Differential privacy**: Extending the audit functor with differential privacy guarantees on aggregated tool usage statistics.

4. **Formal verification**: Machine-checking the security theorems in Lean 4 or Coq, aiming for guarantees comparable to seL4.

11 Conclusion

We have presented a comprehensive theory of the tool interface layer for an AI Operating System. The key contributions are:

1. A **categorical model** (**Tool**) where tools are morphisms, composition is associative (Theorem **??**), capability tokens are representable functors (Definition **??**), and security properties derive from categorical structure.

2. A **formal security architecture** with capability-based access control, sandbox isolation that provably prevents information leakage (Theorem **??**), and contract-locked configurations that freeze the tool set at execution time.

3. A **protocol design** for MCP that achieves universality among tool interfaces (Proposition **??**), with namespacing, parallel discovery, and Bearer token authentication.

4. A **type safety framework** where JSON Schema to Zod conversion is functorial (Definition **??**) and type-safe composition preserves correctness (Theorem **??**).

5. A **production-quality Elixir/OTP implementation** that exploits BEAM process isolation as a natural realisation of categorical sandboxing.

6. **Audit and observability** through a functorial logging system that captures every tool invocation with cost attribution.

The tool interface layer is the bridge between intelligence and action. Just as Unix's device driver model enabled a rich ecosystem of applications by abstracting hardware complexity, the AI OS tool layer enables a rich ecosystem of agents by abstracting the complexity of external system interaction. The categorical framework ensures that this abstraction is principled: composition works, types are preserved, security is enforced, and every action is auditable.

In Part III of this series, we turn to the memory and persistence layer—the mechanism by which agents maintain state across interactions, formalised through the lens of sheaf theory and persistent homology.

References

Anderson, J. P. (1972). Computer security technology planning study. Technical Report ESD-TR-73-51, US Air Force Electronic Systems Division.

Armstrong, J. (2003). *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology, Stockholm.

Awodey, S. (2010). *Category Theory*. Oxford University Press, 2nd edition.

Birgisson, A., Politz, J. G., Erlingsson, Ú., Taly, A., Vrable, M., and Lentczner, M. (2014). Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *NDSS 2014*.

Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. In *NeurIPS 2020*.

Cesarini, F. and Thompson, S. (2009). *Erlang Programming*. O'Reilly Media.

Dennis, J. B. and Van Horn, E. C. (1966). Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155.

E2B (2024). E2B Code Interpreter documentation. https://e2b.dev/docs.

Fong, B. and Spivak, D. I. (2019). *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*. Cambridge University Press.

Hardy, N. (1988). The KeyKOS architecture. *ACM SIGOPS Operating Systems Review*, 19(4):8–25.

Hewitt, C., Bishop, P., and Steiger, R. (1973). A universal modular ACTOR formalism for artificial intelligence. In *IJCAI 1973*, pages 235–245.

Juric, S. (2019). *Elixir in Action*. Manning Publications, 2nd edition.

Klein, G., Elphinstone, K., Heiser, G., et al. (2009). seL4: Formal verification of an OS kernel. In *SOSP 2009*, pages 207–220.

Leinster, T. (2014). *Basic Category Theory*. Cambridge University Press.

Levy, H. M. (1984). *Capability-Based Computer Systems*. Digital Press.

Long, M. (2026a). The AI Operating System, Part I: Kernel architecture—process model, scheduling, and categorical foundations. *GrokRxiv preprint*, 2026.03.

Love, R. (2010). *Linux Kernel Development*. Addison-Wesley, 3rd edition.

Mac Lane, S. (1998). *Categories for the Working Mathematician*. Springer, 2nd edition.

Mac Lane, S. and Moerdijk, I. (1994). *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*. Springer.

Meyerovich, L. A. and Rabkin, A. S. (2013). Empirical analysis of programming language adoption. In *OOPSLA 2013*, pages 1–18.

Miller, M. S., Yee, K.-P., and Shapiro, J. (2003). Capability myths demolished. Technical report, HP Laboratories.

Model Context Protocol Working Group (2024). Model Context Protocol specification, version 1.0. https://modelcontextprotocol.io/specification.

Moggi, E. (1991). Notions of computation and monads. *Information and Computation*, 93(1):55–92.

OpenAI (2024). Function calling — OpenAI API documentation. https://platform.openai.com/docs/guides/function-calling.

Pike, R., Presotto, D., Dorward, S., Flandrena, B., Thompson, K., Trickey, H., and Winterbottom, P. (1995). Plan 9 from Bell Labs. *Computing Systems*, 8(3):221–254.

Ritchie, D. M. and Thompson, K. (1978). The UNIX time-sharing system. *Bell System Technical Journal*, 57(6):1905–1929.

Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2024). Toolformer: Language models can teach themselves to use tools. In *NeurIPS 2023*.

Shapiro, J. S. and Miller, M. S. (2023). Object-capability security: A retrospective. *IEEE Security & Privacy*, 21(2):63–71.

Spivak, D. I. (2014). *Category Theory for the Sciences*. MIT Press.

Tanenbaum, A. S. and Bos, H. (2014). *Modern Operating Systems*. Pearson, 4th edition.

Thomas, D. (2018). *Programming Elixir 1.6*. Pragmatic Bookshelf.

Vercel (2024). Vercel AI SDK documentation, v6. https://sdk.vercel.ai/docs.

Wadler, P. (1995). Monads for functional programming. In *Advanced Functional Programming*, LNCS 925, pages 24–52. Springer.

WASI Working Group (2023). WebAssembly System Interface specification. https://wasi.dev.

Watson, R. N. M., Anderson, J., Laurie, B., and Kennaway, K. (2010). Capsicum: Practical capabilities for UNIX. In *USENIX Security 2010*, pages 29–46.

Xi, H. and Pfenning, F. (1999). Dependent types in practical programming. In *POPL 1999*, pages 214–227.

Yee, K.-P. (2003). Aligning security and usability. *IEEE Security & Privacy*, 2(5):48–55.